Session 5
AADK Task 5

Assessment Report Kotlin Functions, Data Modeling and State Management

Name : Pawankumar
Course : Android App Development with Gen AI

Introduction

Kotlin is a modern programming language widely used for Android development. It supports object■oriented and functional programming paradigms. Kotlin integrates seamlessly with Jetpack Compose to create dynamic and responsive Android applications. This report explores Kotlin concepts through tasks involving functions, data modeling, state management and language comparison.

Task 1
Kotlin Functions, Lambdas & Higher■Order Functions Objective:

Understand Kotlin function types and their advantages in Android development.

Regular Function Example:

```
fun greet(name: String): String { return "Hello " + name
}
```

Lambda Example:

```
val greetLambda: (String) -> String = { name -> "Hello $name"
}
```

Higher Order Function:

```
fun applyOperation(a: Int, b: Int, operation: (Int,Int) -> Int): Int { return operation(a,b)
}
```

Function Style	Example	When to Use	Trade-offs
Regular Function	fun greet()	General logic	More code
Lambda	{ name -> }	Short operations	Less readable when co
Higher Order	fun op(a,b,fn)	Flexible logic	Slight complexity

Task 2
Design a Kotlin Class & Data Model

Feature: Contact Manager A system to store contact details such as name, phone number, email and favorite status.

```
data class Contact( val name:
        String,
        val phone: String?, val email:
        String?,
        val isFavorite: Boolean
)

interface ContactManager {
        fun addContact(contact: Contact)


        fun toggleFavorite(contact: Contact)
}
```

UML Diagram

```
+------------------------ +
|  Contact                |
+------------------------ +
| name                    |
| phone                   |
| email                   |
| isFavorite  |
+------------------------ +
              |
              v
+------------------------------- +
| ContactManager                 |
+------------------------------- +
| addContact()                   |
| toggleFavorite() |
+--------------------------------
```

Task 3
State, Immutability & Side Effects

Immutability ensures reliable UI updates in Compose. Mutable state can lead to unpredictable UI behaviour if not handled carefully.

```
State Flow Diagram User

Action
        | v
   Update State
        | v
     Recomposition Triggered
        | v
UI Updated
```

Task 4
Kotlin vs Java Comparison

Feature	Kotlin	Java	Advantage
Null Safety	Built-in	Manual checks	Safer code
Data Classes	Supported	Manual code	Less boilerplate
Lambdas	Native	Limited	Concise syntax
Extension Functions	Yes	No	Flexible design

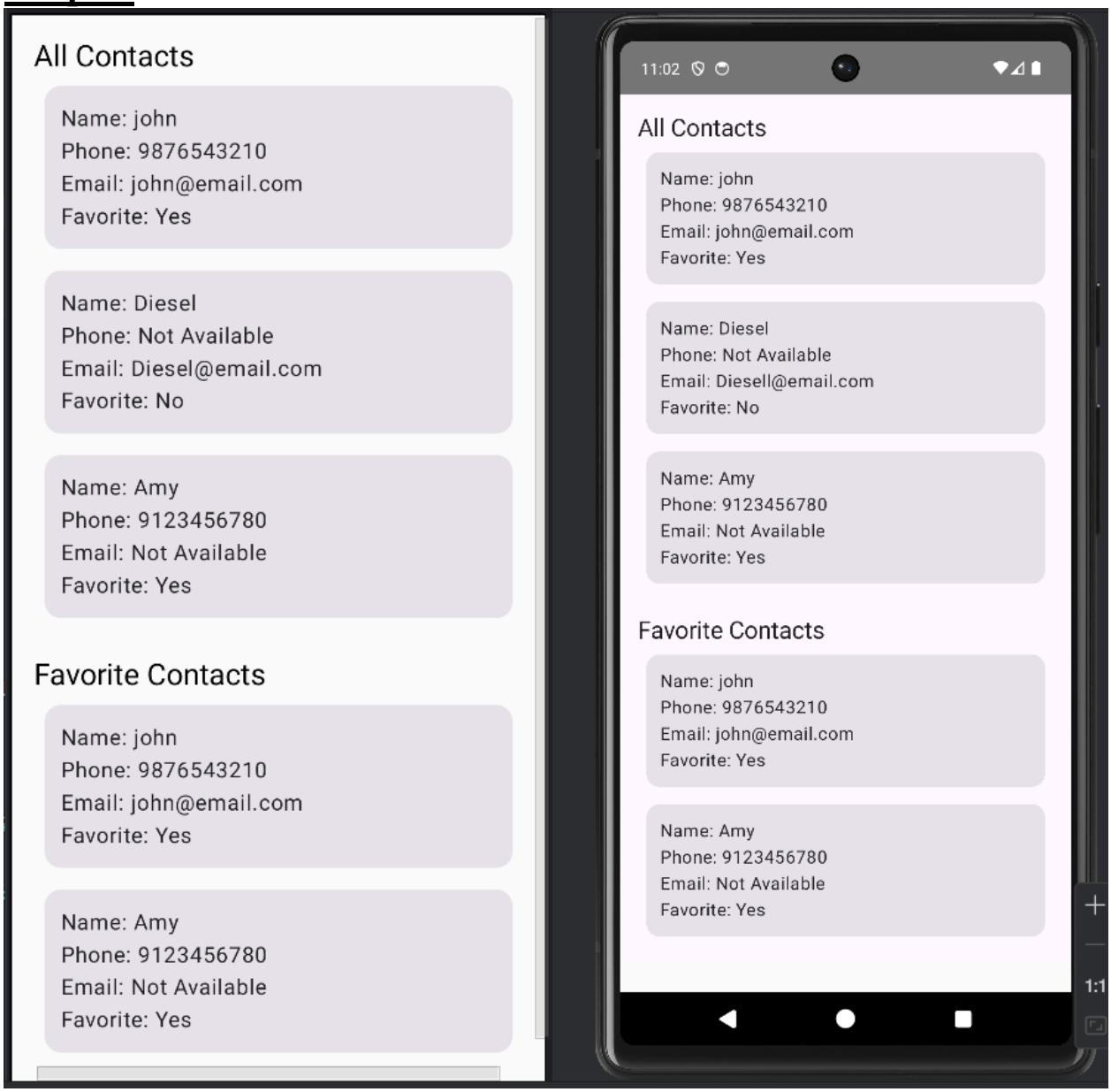
Conclusion

Kotlin simplifies Android development by combining object-oriented and functional programming features. Through tasks exploring functions, data modeling, and state management, developers gain the ability to build scalable and maintainable Android applications using modern frameworks like Jetpack Compose.

Kotlin Code For Contact Manager :

```
package com.example.contactmanager


import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.material3.Card
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Text
import androidx.compose.material3.Surface
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.tooling.preview.Preview
import androidx.compose.ui.unit.dp

// Data class
data class Contact(
    val name: String,
    val phone: String?,
    val email: String?,
    val isFavorite: Boolean
)

// Higher-order function for filtering
fun filterContacts(
    contacts: List<Contact>,
    predicate: (Contact) -> Boolean
): List<Contact> {
    return contacts.filter(predicate)
}

class MainActivity : ComponentActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        val contactList = listOf(
            Contact("john", "9876543210", "john@email.com", true),
            Contact("Diesel", null, "Diesell@email.com", false),
            Contact("Amy", "9123456780", null, true)
        )

        setContent {
            MaterialTheme {
                Surface {
                    ContactScreen(contactList)
                }
            }
        }
    }
}

@Composable
```

```
fun ContactScreen(contacts: List<Contact>) {

    val favorites = filterContacts(contacts) { it.isFavorite }

    Column(modifier = Modifier.padding(16.dp)) {

        Text(
            text = "All Contacts",
            style = MaterialTheme.typography.titleLarge
        )

        ContactList(contacts)

        Spacer(modifier = Modifier.height(20.dp))

        Text(
            text = "Favorite Contacts",
            style = MaterialTheme.typography.titleLarge
        )

        ContactList(favorites)
    }
}

@Composable
fun ContactList(contacts: List<Contact>) {

    LazyColumn {

        items(contacts) { contact ->

            Card(
                modifier = Modifier
                    .fillMaxWidth()
                    .padding(8.dp)
            ) {

                Column(modifier = Modifier.padding(12.dp)) {

                    Text(text = "Name: ${contact.name}")
                    Text(text = "Phone: ${contact.phone ?: "Not Available"}")
                    Text(text = "Email: ${contact.email ?: "Not Available"}")
                    Text(text = "Favorite: ${if (contact.isFavorite) "Yes" else "No"}")

                }
            }
        }
    }
}

@Preview(showBackground = true)
@Composable
fun ContactPreview() {

    val sampleContacts = listOf(
        Contact("john", "9876543210", "john@email.com", true),
        Contact("Diesel", null, "Diesel@email.com", false),
        Contact("Amy", "9123456780", null, true)
    )
```

```
    MaterialTheme {
        ContactScreen(sampleContacts)
    }
}
```

Output: